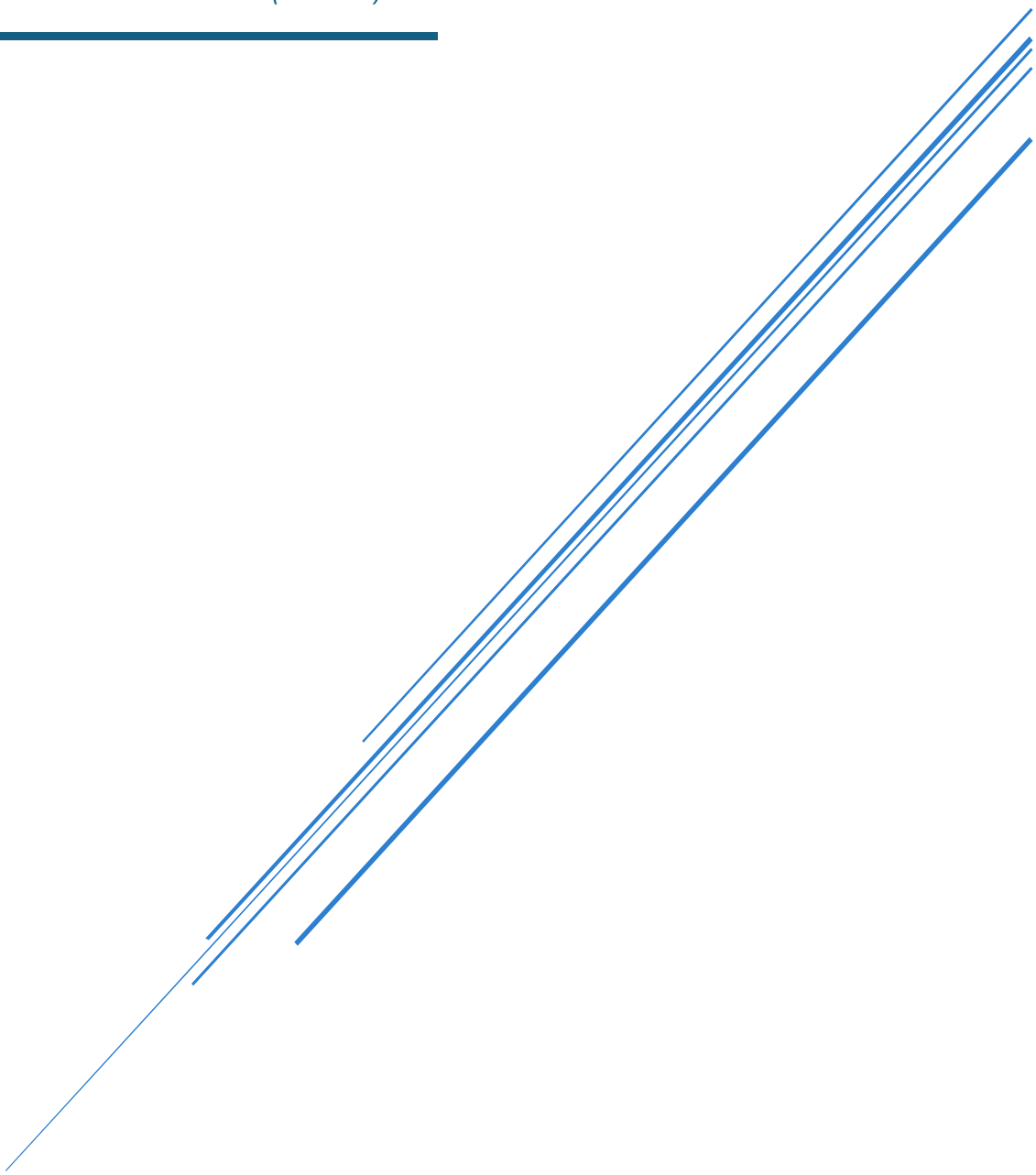
PRG181 – GROUP 3

Project 1

Group Members:	*Student ID:*
Alexander Kruger	*(605298)*
Ruben Botha	*(605467)*
Reitumetse Mdlapo	*(605271)*
Marvin Netshirando	*(605733)*



Belgium Campus Itversity

Date of Submission – 2026/03/18

Contents

Section 1: Array Analysis 2
1D Vs 2D Array 2
Section 2: System Design 3
IPO Table 3
Input 3
Process 4
Output 4
One-Dimensional Array 5
Two-Dimensional Array 5
Index-Tracking 5
Section 3: Error Handling And Validation 6
Department Code Integrity 6
Sale Parameters 6
2D Array Overflow Prevention 6
Output Formatting 6
Section 4: Enhancements 7
Tracking Remaining Transaction Capacity 7
Recording the User Who Entered the Transaction 7
Adding Date and Time Information 7
Comparing Department Performance 7
Saving Data for Future Use 7
Confirmation 8

Section 1: Array Analysis

1D Vs 2D Array

A one-dimensional (1D) array is used to store data in a single list. Each value in the array is stored one after another and can be accessed using one index that shows its position in the array. 1D arrays are useful when you only need to store one group of similar values.

For example, a program could store the scores of five players in a game using a 1D array:

Scores = [120, 95, 140, 110, 130]

Each score can be accessed using its index, such as Scores [0] for the first value or Scores [3] for the fourth value. The main purpose of a 1D array is to keep a simple set of related data organised in one list.

A two-dimensional (2D) array stores data in rows and columns, like a table. This means that two indexes are needed to access a value: one for the row and one for the column. 2D arrays are useful when data needs to be grouped while still storing multiple values for each group.

For example, a program could store students' marks for different tests in a 2D array:

Marks = [

[70, 65, 80],

[55, 60, 72],

[90, 88, 95]

]

In this example, each row represents a student, and each column represents a test mark. A specific value can be accessed using two indexes, such as Marks [1][2].

In this scenario, a 2D array is more suitable because the program needs to store many sales transactions for each department. A two-dimensional array makes it possible to organise the data so that each row represents a department and each column represents a sales transaction. This makes it easier to keep all transactions for a department together.

Using a 2D array also makes it easier for the program to add new sales transactions, keep track of how many transactions each department has, and calculate the total sales for each department at the end of the program.

Section 2: System Design

IPO Table

Input	Processing	Output
Department code Sales amount	1. Store predefined department codes in an array. 2. Check if department code exists in the array. 3. Validates that the sales amount is a number and is greater than zero 4. Records the sales amount in the sales array for the correct department. 5. Updates transaction counter for that specific department 6. Stops transaction entry when the user types END 7. Calculate total sales for each department by summing all array entries for specific department 8. Displays the Final Sales Summary Report showing all the department codes and their sales.	Final Sales Report

Input

The system uses predefined department codes to validate user input. These seven three-character codes are stored in an array of strings. Each code represents a department in the store. Users may enter multiple sales transactions by providing:

-A department code

-A sales amount

The program will keep getting sales data until “END” is entered.

Two main data structures are used:

-A one-dimensional array to store department codes.

-A two-dimensional array to store the sales transactions associated with each department. This design facilitates that the sales data can be arranged and obtained in a systematic way during processing.

Process

The program asks the user to enter the department code and the sales amount, then processes this value to perform several validations checks to ensure data accuracy.

This program will check if given department code is available in the list of valid department codes.

-It checks whether the sales amount is numeric and greater than zero.

-If both inputs are valid, it finds the index position of the required department from the department code array.

-The sales amount is stored in the next available position in the corresponding row in the two-dimensional sales array.

-A transaction counter to record the number of sales entries for each department.

If an invalid department code or an incorrect sales amount is entered, the system will throw an error message and ask for the information to be re-entered.

This function will terminate when the user types “END” and then it will work out the sales report. It then loops through all departments by going through the sales array and calculating sales for each department by summing all values in the sales array.

Output

During operation, the system provides confirmation of data entry in cash register for the particular sales transaction that has been entered and registered.

If invalid data is entered, the system displays clear error messages indicating:

-Invalid department codes

-Non-numeric or negative sales values

-Exceeding the maximum transaction limit (1000 per department)

After all transactions are entered, processed and confirmed, the system will automatically produce a final sales summary report which will include all of the above information.

Each department code

-The total sales amount for each department

-The overall store sales

This output allows users to easily analyse the performance of each department.

One-Dimensional Array

The department codes are stored in a one-dimensional array. The structure is appropriate because the list of departments is fixed and does not change and the list is short. The array of department codes enables a quick search and validation of a department code for each key entered.

Two-Dimensional Array

An array of type two-dimensional is used to store sales transactions. The rows represent departments and the columns represent the sales transactions. Therefore, the system can hold a maximum of 1000 transactions for each department.

Using a two-dimensional array provides several advantages:

-Organised storage of large volumes of sales data

-Efficient retrieval of transactions for specific departments

-Simplified calculation of departmental totals

-Improved data management and readability

Index-Tracking

Counter implemented to track sales transactions for each department. Each counter entry is in a counter array called “salesCount”. The counters are used to find the next empty position in the “sales” array where a sales transaction can be recorded.

Index tracking improves system efficiency by:

-Ensuring that transactions are stored sequentially

-Preventing overwriting of existing records

-Allowing quick access to stored data

-Supporting accurate calculation of department totals

Section 3: Error Handling And Validation

Department Code Integrity

User's input is first validated by parameters then compared to deptCode array. Firstly, user input will be cleaned by removing spaces before and after string, and then if the length of the user's input of department codes is not exactly 3, an error message will be displayed "`Code must be three characters`" and the subroutine will be terminated. Secondly the user's input will be transformed into all caps to avoid any errors caused by case sensitive inputs. Lastly a linear search algorithm will search for a match with user input department code using a Boolean flag and if no match is found the subroutine will be terminated and an error message will appear saying "`Code not found`" to guide the user to their mistake.

Sale Parameters

User's input sale amount is validated through range validation. Input is checked if it is a double to accommodate for decimals and to prevent user from typing numbers as words, for example ten. If the value of the input sale is less than or equal to 0 or if input isn't a double, the system will send an error message to guide the user to their mistake "`Invalid Sale Amount`", and the subroutine will be terminated.

2D Array Overflow Prevention

To ensure that the program doesn't exceed its bounds of 1000 columns, an additional array will be used to track current position as mentioned in Index Tracking. This array (salesCount) will be 1D with a size of 7, Each element represents the last Column position used by the 2D array. This counter will be increased with each input into a department transaction after the counter is validated and the new transaction is added. If salesCount value for the specific department is the final value 999, since modern languages start numbering arrays from 0, then further inputs will be blocked, and an error message will be sent saying "`Transaction Limit Reached`"

Output Formatting

The final Sale Amount is formatted to a currency and to two decimal places as per standard currency formatting improve user readability, and this value must be converted to a string.

Section 4: Enhancements

Tracking Remaining Transaction Capacity

Each department can store up to 1,000 transactions. The program could be improved by displaying the number of remaining transaction slots for each department. This would help the user know how many more sales can still be recorded and avoid exceeding the limit.

Recording the User Who Entered the Transaction

The system could also store the name or ID of the employee who entered each transaction. This would make it easier to track who recorded specific sales and improve accountability within the system.

Adding Date and Time Information

Another improvement would be to record the date and time of each transaction. This would allow the system to track when sales occurred and make it possible to analyse sales trends over different days or times.

Comparing Department Performance

The program could include functionality to compare the total sales of different departments. For example, the system could display which department generated the highest sales and which department had the lowest sales. This would help management evaluate department performance.

Saving Data for Future Use

Currently, the program stores data only while it is running. An improvement would be to save the sales data to a file or database so that it can be accessed again later. This would allow the supermarket to maintain historical records of its sales.

Confirmation

The program could include a user confirmation step after a transaction is entered. After the user inputs the department code and sale amount, the system could display the entered information and ask the user to confirm it before saving the transaction (for example: “Are you sure these details are correct? (Y/N)”). This would help reduce human errors by allowing users to verify the information before it is permanently stored in the system.